

1 Оптимизация вычислений нейронных сетей на GPU

Из roofline модели следует, что есть два основных направления оптимизации вычислений нейронных сетей на GPU: оптимизация вычислений и оптимизация памяти.

Причем, как мы ранее рассматривали, скорость вычисления многих слоёв нейронных сетей на GPU ограничена по памяти (memory-bound сценарий).

Рассмотрим основные методы оптимизации вычислений на GPU, применимые для нейронных сетей.

1.1 Тайлинг в GEMM

Основная операция в нейронных сетях — матричное умножение, GEMM:

$$C = AB,$$

где

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}$$

При наивном вычислении GEMM, мы считаем каждый элемент C как скалярное произведение строки матрицы A на столбец матрицы B .

$$C_{ij} = \sum_{k=1}^K A_{ik} B_{kj}$$

Это приводит к тому, что на одну операцию произведения и одну операцию сложения приходится чтение 8 байтов из GMEM, то есть арифметическая интенсивность такого алгоритма крайне низкая.

Как ранее обсуждалось, при таком подходе одни и те же элементы матриц A и B будут много раз загружаться из глобальной памяти.

Поэтому, снизим количество обращений к глобальной памяти за счёт того, что будем “кэшировать” некоторые элементы в разделяемой памяти, чтобы они были доступны всем потокам блока.

Для этого разобьём вычисление C на блоки меньшего размера $C_{I,J}$, где I — набор строк соответствующего блока, J — набор столбцов. Такой подход называется **тайлинг**.

Этот блок, называемый тайлом, теперь можно представить как сумму произведений тайлов:

$$C_{I,J} = \sum_T A_{I,T} B_{T,J},$$

где T пробегает блоки по внутренней размерности K .

За вычисления одного элемента тайла ответственен один поток, а за вычисление всего тайла $C_{I,J}$ — блок потоков.

Для вычисления одного слагаемого $C_{I,J}^1 = A_{I,1}B_{1,J}$ этой суммы мы можем загрузить тайлы $A_{I,1}$ и $B_{1,J}$ в SMEM. Тогда, чтобы вычислить $C_{I,J}^1$, можно читать данные не из медленной GMEM, а из гораздо более быстрой SMEM. А результат записывать практически бесплатно в RMEM.

Потом мы можем аналогичным образом вычислить $C_{I,J}^2 = A_{I,2}B_{2,J}$. Поскольку каждый поток блока ответственен только за свой элемент тайла $C_{I,J}$, то мы можем просто прибавить результат $C_{I,J}^2$ к $C_{I,J}^1$.

Так, пока в цикле выполняются вычисления $C_{I,J}^t$, каждый поток может хранить в RMEM кумулятивную сумму элемента $C_{I,J}$, за который он ответственен.

Арифметическая интенсивность тайлинга Выглядит все так, будто мы лишь раздули количество операций чтения и записи, а значит снизили арифметическую интенсивность.

Так и есть. Но только формально.

Вот арифметическая интенсивность для наивного GEMM:

$$I_{\text{naive}} = \frac{2 \text{ FLOP}}{2 \cdot 4 \text{ bytes}} = \frac{1}{4} \text{ FLOP/byte}$$

Посчитаем арифметическую интенсивность для тайлинга. Отдельно оценим обращения к GMEM и SMEM. Число операций для вычисления $C_{I,J}^t$ равно:

$$\text{FLOP}_{\text{tile}} = 2B_M B_N B_K,$$

где:

- B_M — размер тайла по размерности M ,
- B_N — размер тайла по размерности N ,
- B_K — размер тайла по размерности K .

Из GMEM нужно прочесть два тайла:

$$\text{Bytes}_{\text{GMEM}} = 4(B_M B_K + B_K B_N)$$

При этом из SMEM потоки читают элементы тайлов $A_{I,T}$ и $B_{T,J}$ для вычисления своих элементов $C_{I,J}$. Каждый поток читает B_K элементов из тайла A и B_K элементов из тайла B . Тогда:

$$\text{Bytes}_{\text{SMEM}} = 2 \cdot 4B_M B_N B_K$$

Тогда арифметическая интенсивность будет равна

$$I_{\text{tiled}} = \frac{2B_M B_N B_K}{\text{Bytes}_{\text{GMEM}} + \text{Bytes}_{\text{SMEM}}} = \frac{2B_M B_N B_K}{4(B_M B_K + B_K B_N) + 2 \cdot 4B_M B_N B_K}$$

Если упростить, то

$$I_{\text{tiled}} = \frac{1}{2/B_N + 2/B_M + 4} \leq \frac{1}{4} \text{ FLOP/byte}$$

Видно, что арифметическая интенсивность тайлинга всегда будет хуже, чем арифметическая интенсивность наивного подхода.

Однако, это утверждение было бы верно только в том случае, если бы стоимость обращений к GMEM была равна стоимости обращений к SMEM.

На практике же, в современных архитектурах GPU задержка обращения к SMEM может быть на 2 порядка меньше, чем задержка обращения к GMEM. Поэтому при подсчёте арифметической интенсивности, мы можем не учитывать обращения к SMEM. А значит

$$I_{\text{tiled}}^{\text{GMEM}} = \frac{2B_M B_N B_K}{4(B_M B_K + B_K B_N)} = \frac{1}{2/B_N + 2/B_M}$$

Таким образом:

$$I_{\text{naive}} \leq I_{\text{tiled}}^{\text{GMEM}}$$

и равенство достигается только в том случае, если $B_N = 1$ и $B_M = 1$ (то есть когда тайлинг по сути отсутствует).

1.2 Повышение арифметической интенсивности

Из выражения

$$I_{\text{tiled}}^{\text{GMEM}} = \frac{1}{2/B_N + 2/B_M}$$

можно сделать вывод, что чем больше B_N и B_M , то есть размерности тайла матрицы C , тем выше арифметическая интенсивность. А значит, чтобы ускорить вычисление GEMM, нужно увеличивать размер тайла.

Ранее мы выполняли GEMM с расчётом, что каждый поток ответственен за свой элемент итоговой матрицы, а блок потоков — за тайл. Если мы будем увеличивать размерность итогового тайла, то упрёмся в максимальный размер блока: 1024 потока.

Чтобы перешагнуть через этот блок, пусть каждый поток считает не один элемент, а небольшой набор — свой подтайл (subtile). Кумулятивные суммы в таком случае будут храниться в виде массива в RMEM.

Таким образом, итоговый тайл становится больше, а число потоков остаётся тем же. Однако, такое решение не является бесплатным, поскольку увеличивается регистровое давление и потребление SMEM. Это в свою очередь может привести к уменьшению оверсапу.

1.3 Вычисления в пониженной точности

Мы увеличили арифметическую интенсивность посредством уменьшения количества обращений к GMEM. Однако, в формуле арифметической интенсивности

$$I_{\text{tiled}}^{\text{GMEM}} = \frac{1}{2/B_N + 2/B_M},$$

вернее, в немного другой ее версии

$$I_{\text{tiled}}^{\text{GMEM}} = \frac{2}{4(1/B_N + 1/B_M)}$$

можно заметить константу 4 — это количество байт, необходимое для хранения одного числа.

Обычно, при обучении нейронных сетей используются числа с плавающей точкой. Базовый формат — `float32`, где 32 означает число бит, то есть 4 байта. Такое число представляется следующим образом:

- 1 бит — знак s ,
- 8 бит — показатель степени e ,
- 23 бита — мантисса m .

Нормализованное значение (то есть не описывающее специальные случаи, такие как 0, NaN, inf и тд) вычисляется по формуле

$$(-1)^s \cdot 2^{e-\text{bias}} \cdot 1.m,$$

где для `float32` $\text{bias} = 127$, а $1.m$ означает, что перед дробной частью мантиссы неявно стоит ведущая единица (таким образом мы не храним ее явно, что позволяет использовать еще один бит точности).

Но, на практике такая высокая точность оказывается избыточной для некоторых операций нейронных сетей. Значения весов чаще всего лежат около нуля, а небольшая погрешность при их изменении может не оказывать существенного влияния на результат.

Разумным решением будет снизить точность и использовать для хранения весов тип `float16`. Такое число будет представлено в памяти уже следующим образом:

- 1 бит — знак,
- 5 бит — показатель степени,
- 10 бит — мантисса

Уменьшение количества байт на параметр сети само собой также уменьшит количество требуемой для модели памяти. А больший объем памяти в свою очередь позволит обучать большие модели.

Однако веса — не единственные тензоры, которые можно хранить в пониженной точности. Во время обучения значительный объем памяти занимают также активации.

Активации нужны не только во время вычисления прямого прохода — они также участвуют в вычислении обратного распространения ошибки: после вычисления слоя их приходится сохранять, поскольку они необходимы для вычисления градиентов. Объем памяти, занимаемый активациями, может превосходить объем памяти, занимаемый весами.

Таким образом, уменьшив количество байтов, необходимых для хранения данных, вдвое, мы увеличим вдвое арифметическую интенсивность. В частности, арифметическую интенсивность матричного умножения:

$$I_{\text{tiled, fp16}}^{\text{GMEM}} = \frac{2}{2(1/B_N + 1/B_M)} = \frac{1}{1/B_N + 1/B_M}$$

1.4 Переполнение и округление значений с плавающей точкой

Но у `float16` есть важный недостаток: у него меньше бит под показатель степени. Поэтому диапазон представимых значений существенно уже, чем у `float32`:

$$\text{float32: } \approx 1.18 \cdot 10^{-38} \dots 3.4 \cdot 10^{38},$$

$$\text{float16: } \approx 6.10 \cdot 10^{-5} \dots 6.55 \cdot 10^4$$

`float16` хуже представляет большие числа. Конечно, параметры модели в большинстве своем лежат в интервале $(-1; 1)$. Но при вычислении, например, суммы или экспоненты, возможно, что итоговое значение окажется вне этого диапазона. Если оно будет храниться в `float16`, то случится **переполнение типа с плавающей точкой (float overflow)**.

Чтобы избежать подобной ситуации, был разработан тип данных `bfloat16`, где:

- 1 бит — знак;
- 8 бит — показатель степени;
- 7 бит — мантисса.

Поскольку `bfloat16` сохраняет столько же бит под показатель степени, что и `float32`, то и диапазон значений у него будет близок к диапазону `float32`.

Цена этого — меньшая мантисса. Вещественная числовая прямая непрерывна, однако представление чисел конечным количеством бит дискретизирует множество значений, которые мы можем хранить.

В числах с плавающей точкой степень дискретизации зависит от масштаба числа и от количества бит мантиссы. Чем больше число — тем большая будет погрешность при округлении к наиболее близкому представлению, поскольку чем больше масштаб, тем больше становится расстояние между ближайшими числами, представимыми в такой записи. А это расстояние уже напрямую зависит от точности мантиссы.

Это приводит к тому, что при выполнении арифметики с большими по модулю числами, мы будем накапливать все большую и большую погрешность.

Уже при хранении данных в 16 битах у нас возникают проблемы с диапазоном и точностью. Но это не значит, что мы на этом остановимся! Существуют архитектуры GPU, поддерживающие аппаратные вычисления в 8-битной точности и в даже 4-битной точности. Однако, при таких вычислениях нам необходимо применять некоторые хитрости, о которых мы поговорим позднее.

1.5 Тензорные ядра в GEMM

Рассмотрим арифметическую интенсивность вычисления GEMM в `bfloat16` и с тайлингом, блоками 128×128 элементов, где каждый поток считает участок в 4×4 элементов:

$$I_{\text{tiled, bf16}}^{\text{GMEM}} = \frac{1}{1/128 + 1/128} = 64$$

Это достаточно высокий результат. Возможно теперь стоит сместить фокус внимания с оптимизации памяти на оптимизацию вычислений? Действительно, в погоне за арифметической интенсивностью мы заставили один поток считать сразу 16 элементов, что, откровенно говоря, ломает саму идею GPU — параллелизм вычислений.

Как мы ранее рассматривали, в современных GPU для матричных умножений есть специальные аппаратные блоки — тензорные ядра. Они предназначены для быстрого выполнения операций матричного умножения и аккумуляции результата на небольших матричных блоках:

$$C_{I,J}^R \leftarrow C_{I,J}^R + A_{I,T}^R B_{T,J}^R,$$

здесь $C_{I,J}^R$, $A_{I,T}^R$ и $B_{T,J}^R$ — блоки, хранящиеся в памяти регистров. То есть работа с данными остаётся прежней:

$$\text{GMEM} \rightarrow \text{SMEM} \rightarrow \text{RMEM},$$

Однако, вычисления смещаются с уровня потоков на уровень варпов. То есть вычисления внутри регистрового тайла выполняют не CUDA-ядра,

последовательно находя значения каждого из, например, 16 элементов. Теперь фрагмент блока, за который ответственен один **варп**, вычисляется параллельно с помощью тензорных ядер.

Тензорные ядра наиболее эффективны для вычислений в пониженной точности, аккумулируя при этом значения в полной точности, чтобы минимизировать накапливаемую погрешность.

Арифметическая интенсивность при этом не меняется. Зато, с точки зрения гоofline-модели, тензорные ядра позволяют поднять вычислительный потолок. Так что если операция была compute-bound, то использование тензорных ядер может дать большое ускорение (в отличие от memory-bound сценариев).

При этом тензорные ядра не ускоряют любую операцию в нейронной сети. Они предназначены прежде всего для матричных операций или близких к ним. Но поскольку GEMM — одна из основных операций в нейронных сетях, тензорные ядра могут быть эффективны при вычислении:

- FFN-модулей;
- механизма внимания;
- свёрток

но вряд ли будут полезны при выполнении редукций или поэлементных операций.

1.6 Слияние кернелов (kernel fusion)

Такие оптимизации, как тайлинг, повышающий арифметическую интенсивность, и использование тензорных ядер, не применимы к поэлементным операциям и нормализациям.

Основная проблема этих операций — слишком низкая арифметическая интенсивность, поскольку каждый раз необходимо загружать данные из глобальной памяти.

Однако, слои нейронной сети, в основном, выполняются последовательно. И те результаты, которые мы получили на предыдущем слое, необходимо загрузить в глобальную память. А для следующего слоя, эти же самые результаты необходимо снова достать из глобальной памяти.

При выполнении кернела требуется сохранить результат в глобальную память в конце его работы, иначе этот результат исчезнет. Поэтому, чтобы избежать лишних перемещений данных, пусть один кернел ответственен за вычисление сразу нескольких слоёв.

Например, кернел, в котором мы выполняли матричное умножение, пусть до выгрузки результата в GMEM, например, прибавит к этому результату смещение и применит функцию активации.

Это уменьшает проблему низкой арифметической интенсивности таких операций, поскольку они теперь либо требуют значительно меньшего чтения данных из GMEM, либо не требуют их вовсе.

Этот подход называется **слиянием ядер** (**kernel fusion**).

Однако, слияние ядер — не бесплатная оптимизация. Объединение слишком большого количества операций может увеличить регистровое давление и давление на SMEM, что в свою очередь может привести к падению оверхеда.

К сожалению, по этой причине мы не можем сделать один большой ядро для всей модели. Однако слияние ядер крайне полезно в следующих сценариях (в том числе после GEMM):

- несколько последовательных поэлементных операций подряд;
- последовательные операции смещения и применения функции активации;
- последовательные операции применения остаточных связей и нормализации

Также мы можем объединить операции механизма внимания в один ядро. Такой ядро называется **Flash Attention**.

2 Flash Attention

2.1 Проблемы наивного внимания

Рассмотрим одну голову внимания. Механизм внимания внутри этой головы вычисляется следующим образом:

$$A = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right),$$

$$O = AV$$

При наивной реализации, вычисление внимания будет разбито на несколько этапов:

1. Вычислить $S = QK^T / \sqrt{d}$, после чего результат записать в GMEM;
2. Вычислить $A = \text{Softmax}(S)$, после чего результат также записать в GMEM;
3. Вычислить $O = AV$, после чего результат также записать в GMEM.

Кроме постоянного чтения и записи данных в GMEM, которые как мы уже обсуждали, очень дорогие, есть проблема гораздо серьезнее: материализация матриц S и A в глобальной памяти.

Эти матрицы имеют размер $N \times N$, где N — длина обрабатываемой последовательности токенов. Таким образом, нам необходимо хранить $2 \cdot N^2$

активаций. Если длина последовательности большая, например 10^5 элементов, то нам придется хранить только на одном слое $2 \cdot 10^{10}$ значений, которые займут примерно 40 гигабайт памяти. То есть для трансформера со всего восемью слоями, только эти *промежуточные* активации будут занимать объем памяти, равный всему объему памяти четырех GPU H100 по 80 гигабайт.

2.2 Идея Flash Attention

Flash Attention, в свою очередь, не материализует матрицы S и A . Вместо этого воспользуемся идеей тайлинга: будем вычислять итоговую матрицу по O по блокам, при этом объединив все операции в один кернел.

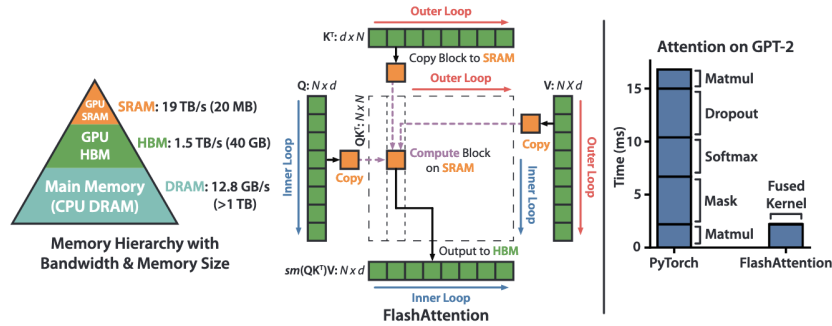


Рис. 1: Схема Flash Attention.

Такой подход позволит не только избавиться от хранения промежуточных активаций, но и применить оптимизации, которые мы рассматривали ранее: 'кэшировать' данные в SMEM и использовать тензорные ядра.

2.3 Численно стабильный Softmax (Safe Softmax)

Перед тем, как перейти к блочному вычислению внимания, разберем вычисление Softmax.

В обычном случае мы применяем Softmax к одной строке следующим образом:

$$\text{Softmax}(x)_i = \frac{\exp(x_i)}{\sum_{j=1}^N \exp(x_j)}$$

Мы бы хотели проводить вычисления в пониженной точности — в **bf16** типе. Однако, ранее мы пришли к выводу, чем больше масштаб значений, тем выше погрешность вычислений. Наиболее неустойчивы операции к таким погрешностям — вычисление экспоненты и суммирование длинных последовательностей.

К сожалению, так совпало, что при вычислении Softmax нам необходимо делать и то, и другое одновременно — вычислить сумму экспонент.

Если же мы будем использовать `fp16` тип, то рискуем получить переполнение.

Чтобы избежать этих проблем, используется **численно стабильная версия Softmax (Safe Softmax)**. Воспользуемся инвариантностью Softmax к сдвигу:

$$\text{Softmax}(x - m)_i = \frac{\exp(x_i - m)}{\sum_{j=1}^N \exp(x_j - m)} = \frac{\exp(-m) \exp(x_i)}{\exp(-m) \sum_{j=1}^N \exp(x_j)} = \frac{\exp(x_i)}{\sum_{j=1}^N \exp(x_j)},$$

то есть

$$\text{Softmax}(x)_i = \text{Softmax}(x - m)_i$$

Чтобы избежать большого масштаба элементов, будем вычитать из каждого элемента максимум по строке:

$$m = \max_j x_j$$

таким образом, все элементы становятся неположительными, а значит значения экспонент будут находиться в полуинтервале $(0; 1]$.

В наивной реализации это сделать просто, потому что вся строка S_i уже материализована в GMEM. Мы можем пройти по ней, найти максимум, затем еще раз пройти и посчитать сумму экспонент.

Однако, во Flash Attention мы не хотим считать всю строку целиком, мы хотим обрабатывать матрицы по блокам. Поэтому обычного safe softmax недостаточно: нужно научиться пересчитывать максимум и сумму экспонент постепенно, по мере обработки блоков.

2.4 Online Softmax

Во Flash Attention используется версия численно стабильного Softmax, которую удобно применять при тайлинге — **online-softmax**.

Пусть строка x разбита на несколько блоков:

$$x = [x^{(1)}, x^{(2)}, \dots, x^{(T)}]$$

Нам нужно посчитать softmax по всей строке x , но при этом не хранить всю строку целиком. Для этого будем обрабатывать блоки последовательно.

Для численно стабильного softmax нам нужны две величины:

$$m = \max_j x_j, \quad l = \sum_j \exp(x_j - m)$$

Тогда:

$$\text{Softmax}(x)_i = \frac{\exp(x_i - m)}{l}.$$

В online-softmax мы будем поддерживать эти величины постепенно. Пусть после обработки $t - 1$ блоков у нас уже есть максимум $m^{(t-1)}$ среди уже обработанных элементов, и

$$l^{(t-1)} = \sum_{\text{old}} \exp(x_j - m^{(t-1)}),$$

сумма экспонент, посчитанная относительно этого максимума.

Обработаем новый блок $x^{(t)}$. Сначала найдем максимум внутри этого блока:

$$m_t = \max_j x_j^{(t)}$$

Новый максимум по всем обработанным элементам будет равен

$$m^{(t)} = \max(m^{(t-1)}, m_t)$$

Теперь нужно обновить сумму экспонент. Посчитаем сумму по блоку с новым максимумом:

$$\hat{l}^{(t)} = \sum_{x_j \in x^{(t)}} \exp(x_j - m^{(t)})$$

Но старую сумму нельзя просто сложить с суммой по новому блоку, потому что старые значения были нормированы относительно другого максимума. Поэтому нужно перенормировать значения

$$l^{(t)} = \frac{\exp(m^{(t-1)})}{\exp(m^{(t)})} l^{(t-1)} + \hat{l}^{(t)}$$

или если упростить

$$l^{(t)} = \exp(m^{(t-1)} - m^{(t)}) l^{(t-1)} + \hat{l}^{(t)}$$

Теперь мы можем применить softmax к тайлингу механизма внимания.

2.5 QKV-тайлинг

Разобьем матрицы Q , K и V на блоки. Пусть мы хотим вычислить блок строк O_i , соответствующий блоку строк запросов Q_i . Для этого нужно учесть все блоки ключей и значений:

$$(K_1, V_1), (K_2, V_2), \dots, (K_T, V_T)$$

На каждой итерации мы берем текущий блок K_t и вычисляем блок матрицы оценок внимания $S_{i,t}$:

$$S_{i,t} = \frac{Q_i K_t^T}{\sqrt{d}}.$$

Далее сразу используем этот блок для обновления Softmax. Найдём максимумы по строкам:

$$m_t = \max_j S_{i,t}^{(:,j)}$$

m_t в данном случае — не число, а вектор. Обновим максимум:

$$m^{(t)} = \max(m^{(t-1)}, m_t).$$

Затем посчитаем сумму экспонент по текущему блоку относительного нового максимума:

$$\hat{l}^{(t)} = \sum_j \exp(S_{i,t}^{(j)} - m^{(t)})$$

и обновим сумму экспонент:

$$l^{(t)} = \exp(m^{(t-1)} - m^{(t)})l^{(t-1)} + \hat{l}^{(t)},$$

$l^{(t)}$ — тоже вектор.

В механизме внимания нам нужна не сама матрица весов A , а результат умножения на V :

$$O_i = A_i V$$

Матрицу весов внимания можно переписать таким образом:

$$A_{ij} = \frac{\exp(S_{ij} - m_i)}{\sum_k \exp(S_{ik} - m_i)},$$

где знаменатель — наша накапливаемая сумма экспонент. То есть

$$A_{ij} = \frac{\exp(S_{ij} - m_i)}{l^{(t)}},$$

Не будем сразу делить на $l^{(t)}$ — всё-таки, это не конечный результат. Поэтому вместе с m и l будем поддерживать ненормированную сумму текущего блока $U^{(t)}$, где значение по старым блокам равно

$$U^{(t-1)} = \sum_{\text{old } j} \exp(S_{ij} - m^{(t-1)})V_j$$

$U^{(t)}$ напоминает $l^{(t)}$. Вычислим для текущего блока вклад в эту сумму:

$$\hat{U}^{(t)} = \exp(S_{i,t} - m^{(t)})V_t$$

Тогда старую сумму нужно нормировать относительно нового максимума:

$$U^{(t)} = \exp(m^{(t-1)} - m^{(t)})U^{(t-1)} + \hat{U}^{(t)}.$$

Тогда после обработки всех блоков итоговый выход получается равным:

$$O_i = \frac{U^{(T)}}{l(T)}$$

В глобальную память записываем только результат O_i — несколько строк итоговой матрицы.

2.6 Сравнение наивного внимания и Flash Attention

Важно, что Flash Attention не меняет математическую формулу внимания. Мы по-прежнему считаем

$$O = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

только последовательность действий не как в наивном механизме внимания, где мы выполняем все эти шаги по порядку. Вместо этого, каждый блок потоков сразу считает O_i , постепенно обновляя результат, по мере прохождения по блокам ключей и значений

$$(K_1, V_1), (K_2, V_2), \dots, (K_T, V_T).$$

2.7 Маскированное внимание в Flash Attention

Пока мы рассмотрели только полное внимание — когда каждый токен может смотреть на каждый другой токен. Однако на практике почти всегда используется маска внимания.

В наивной реализации маска применяется к матрице оценок внимания S перед softmax. Если элемент запрещен маской, то ему присваивается значение $-\infty$. Тогда после softmax этот элемент получит нулевой вес, а значит не будет участвовать во взвешенной сумме

$$O_i = \sum_j A_{ij} V_j$$

Во Flash Attention полной матрицы S нет. Есть только текущий блок:

$$S_{i,t} = \frac{Q_i K_t^T}{\sqrt{d}}$$

Поэтому маску нужно применять сразу к блоку $S_{i,t}$ до обновления online-softmax.

Обозначим замаскированный блок через $\tilde{S}_{i,t}$, где каждый элемент:

$$\tilde{s}_{i,t} = \begin{cases} s_{i,t}, & \text{если позиция разрешена,} \\ -\infty, & \text{если позиция запрещена} \end{cases}$$

Дальше все вычисления выполняются не с $S_{i,t}$, а с $\tilde{S}_{i,t}$. Поскольку $\exp(-\infty) = 0$ и $-\infty$ точно не будет больше максимума, то замаскированные элементы не влияют ни на максимум, ни на сумму экспонент, ни на $U^{(t)}$.

В авторегрессионной задаче мы маскируем все элементы выше главной диагонали. При блочном вычислении для каждого такого блока важно понимать, где он расположен:

- если блок полностью ниже главной диагонали, то все элементы разрешены, мы можем считать его как обычный блок внимания;
- если блок полностью выше главной диагонали — все элементы запрещены, его можно не считать вообще;
- если блок пересекает диагональ — часть элементов разрешена, а часть запрещена, значит необходимо применить маску.

Возникает вопрос: но ведь мы считаем блок из нескольких строк матрицы O_i . Но для его вычисления мы всё равно идём по блокам ключей K_t , и блокам значений V_t . На каждой итерации возникает прямоугольный блок оценок внимания. И если этот блок полностью замаскирован — значит мы можем не проводить вычисления с соответствующими K_t и V_t .

2.8 Обратное распространение в Flash Attention

Все это время мы рассматривали только kernel, соответствующий прямому распространению в механизме внимания. Теперь необходимо взяться за самую сложную часть. Механизм внимания необходимо обучать, а значит нам нужен алгоритм Flash Attention для обратного распространения.

В наивной реализации выполнить обратное распространение в механизме внимания было достаточно просто, потому что во время прямого прохода мы материализовали матрицу весов внимания A .

Проблема в том, что Flash Attention не сохраняет матрицу A в глобальной памяти, а значит мы не можем использовать ее для обратного распространения.

Поэтому давайте пересчитаем нужные блоки матрицы A . Пусть мы временно сохранили необходимые статистики m_i и l_i во время прямого прохода.

При обратном распространении мы снова берем блок строк Q_i , а также текущие блоки ключей и значений (K_t, V_t) . Затем пересчитываем блок оценок внимания:

$$S_{i,t} = \frac{Q_i K_t^T}{\sqrt{d}}$$

Если используется маска, то применяем ее также, как и при прямом распространении:

$$S_{i,t} \rightarrow \tilde{S}_{i,t}$$

После этого можно восстановить соответствующий блок матрицы A :

$$A_{i,t} = \frac{\exp(\tilde{S}_{i,t} - m_i)}{l_i}$$

Матрица A не хранится в памяти целиком, но отдельные её блоки мы восстанавливаем по мере необходимости.

Пусть со следующего слоя к механизму внимания пришла производная функции потерь по выходу $\frac{\partial L}{\partial O_i}$. Начнем с выражения

$$O_i = A_i V$$

Для одной строки O_i можно записать

$$O_{ic} = \sum_j A_{ij} V_{jc},$$

где c — индекс компоненты вектора значения. Попробуем найти производную функции потерь по элементу V_{jc} :

$$\frac{\partial L}{\partial V_{jc}} = \sum_i \frac{\partial L}{\partial O_{ic}} \frac{\partial O_{ic}}{\partial V_{jc}}$$

Поскольку

$$\frac{\partial O_{ic}}{\partial V_{jc}} = A_{ij},$$

то получаем, что

$$\frac{\partial L}{\partial V_{jc}} = \sum_i A_{ij} \frac{\partial L}{\partial O_{ic}}$$

или же в матричном виде:

$$\frac{\partial L}{\partial V_t} = A_{i,t}^T \frac{\partial L}{\partial O_i}$$

Один и тот же блок V_t участвует в вычислении разных блоков выхода O_i , поэтому производная по V_t накапливает вклады от разных блоков строк Q_i .

Теперь найдем производную по весам внимания $A_{i,t}$. Из того же выражения

$$O_{ic} = \sum_j A_{ij} V_{jc}$$

получаем

$$\frac{\partial O_{ic}}{\partial A_{ij}} = V_{jc}$$

Тогда:

$$\frac{\partial L}{\partial A_{ij}} = \sum_c \frac{\partial L}{\partial O_{ic}} V_{jc}$$

или же в матричном виде:

$$\frac{\partial L}{\partial A_{i,t}} = \frac{\partial L}{\partial O_i} V_t^T$$

Теперь нужно пройти через Softmax. Для одной строки производная функции потерь по элементу S_{ij} записывается следующим образом:

$$\frac{\partial L}{\partial S_{ij}} = A_{ij} \left(\frac{\partial L}{\partial A_{ij}} - \sum_k \frac{\partial L}{\partial A_{ik}} A_{ik} \right)$$

На первый взгляд кажется, что для вычисления суммы $\sum_k \frac{\partial L}{\partial A_{ik}} A_{ik}$ нам снова потребуется вся строка A_i . Но это выражение можно переписать через уже известные значения O_i и $\frac{\partial L}{\partial O_i}$:

$$\sum_j \frac{\partial L}{\partial A_{ij}} A_{ij} = \sum_j \left(\sum_c \frac{\partial L}{\partial O_{ic}} V_{jc} \right) A_{ij}$$

Поменяем порядок суммирования:

$$\sum_j \frac{\partial L}{\partial A_{ij}} A_{ij} = \sum_c \frac{\partial L}{\partial O_{ic}} \left(\underbrace{\sum_j A_{ij} V_{jc}}_{O_{ic}} \right) = \sum_c \frac{\partial L}{\partial O_{ic}} O_{ic}$$

Обозначим эту величину через D_i . Если O_i — это блок строк, то D_i не число, а вектор: по одному значению на каждую строку блока.

Теперь производную по блоку оценок внимания можно записать так:

$$\frac{\partial L}{\partial S_{i,t}} = A_{i,t} \odot \left(\frac{\partial L}{\partial A_{i,t}} - D_i \right)$$

Осталось найти производные по Q_i и K_t . Для элемента матрицы оценок внимания верно

$$S_{ab} = \frac{1}{\sqrt{d}} \sum_c (Q_i)_{ac} (K_t)_{bc}$$

Найдем производную по элементу $(Q_i)_{ac}$:

$$\frac{\partial S_{ab}}{\partial (Q_i)_{ac}} = \frac{(K_t)_{bc}}{\sqrt{d}}$$

Тогда

$$\frac{\partial L}{\partial (Q_i)_{ac}} = \sum_b \frac{\partial L}{\partial S_{ab}} \frac{(K_t)_{bc}}{\sqrt{d}}$$

или в матричном виде:

$$\frac{\partial L}{\partial Q_i} += \frac{1}{\sqrt{d}} \frac{\partial L}{\partial S_{i,t}} K_t$$

Аналогично для K_t :

$$\frac{\partial L}{\partial K_t} += \frac{1}{\sqrt{d}} \left(\frac{\partial L}{\partial S_{i,t}} \right)^T Q_i$$

На первый взгляд это может показаться странным: мы специально пересчитываем то, что уже считали во время прямого распространения.

Однако, на практике оказывается, что на запись в глобальную память и чтение из нее промежуточных результатов уходит больше времени, чем на пересчитывание такого блока. Такая идея называется **activation checkpointing**.